LAPORAN PRAKTIKUM

PRAKTIKUM PENGUJIAN PERANGKAT LUNAK

PERTEMUAN 4

MOCK, SPY, DAN STUB TESTING MENGGUNAKAN MOCKITO

Disusun oleh:

Nama	: Januarsyah Akbar
NIM	: 24/535846/SV/24314
Kelas	: PL4B2
Dosen Pengampu	: Divi Galih Prasetyo Putri, S.Kom., M.Kom., Ph.D.

PROGRAM STUDI D-IV

TEKNOLOGI REKAYASA PERANGKAT LUNAK

DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA

SEKOLAH VOKASI

UNIVERSITAS GADJAH MADA

YOGYAKARTA

March 12, 2026

Daftar Isi

Daftar Isi **2**

Tujuan Praktikum **3**

Hasil dan Pembahasan **4**
2.1 Langkah Praktikum 4
2.1.1 Kelas Pendukung 4
2.1.2 Mock Test 5
2.1.3 Spy Test 6
2.1.4 Stub Test 7
2.2 Tugas 9
2.2.1 Kelas-Kelas Book 9
2.2.2 BookTest 10

Resources **14**

Kesimpulan **15**

Daftar Pustaka **16**

Tujuan Praktikum

1. Mahasiswa mampu memahami konsep dan perbedaan antara Mock, Spy, dan Stub dalam pengujian perangkat lunak.
2. Mahasiswa mampu mengimplementasikan Mock, Spy, dan Stub menggunakan framework Mockito pada proyek Java.
3. Mahasiswa mampu melakukan verifikasi interaksi dan verifikasi state dalam unit test menggunakan anotasi dan metode Mockito.

Hasil dan Pembahasan

2.1 Langkah Praktikum

Pada langkah praktikum ini, dilakukan implementasi tiga teknik pengujian berbasis Mockito, yaitu **Mock**, **Spy**, dan **Stub**. Sebelum menjalankan pengujian, disiapkan terlebih dahulu kelas pendukung `User`, `UserRepository`, dan `UserService`.

2.1.1 Kelas Pendukung

Kelas `User` merupakan model data pengguna dengan atribut `id` dan `name`.

```java
public class User {
    public String id;
    public String name;

    public User(String id, String name) {
        this.id = id;
        this.name = name;
    }
}
```

Listing .1: User.java

`UserRepository` adalah antarmuka (interface) yang menyediakan operasi `save` dan `findById`.

```java
public interface UserRepository {
    void save(User user);
    User findById(String id);
}
```

Listing .2: UserRepository.java

`UserService` berisi logika bisnis, yaitu mendaftarkan pengguna dan mengambil nama pengguna berdasarkan ID.

```java
public class UserService {
    private final UserRepository repository;

    public UserService(UserRepository repository) {
        this.repository = repository;
    }

    public void register(User user) {
        repository.save(user);
    }

```

```java
    public String getUserName(String id) {
        User user = repository.findById(id);
        return user.name;
    }
}
```

Listing .3: UserService.java

2.1.2 Mock Test

Mock adalah objek tiruan yang sepenuhnya dikendalikan oleh framework Mockito. Seluruh metodenya mengembalikan nilai default dan tidak menjalankan implementasi nyata. Mock digunakan untuk *verifikasi interaksi*, yaitu memastikan bahwa metode tertentu dipanggil (atau tidak dipanggil) selama eksekusi.

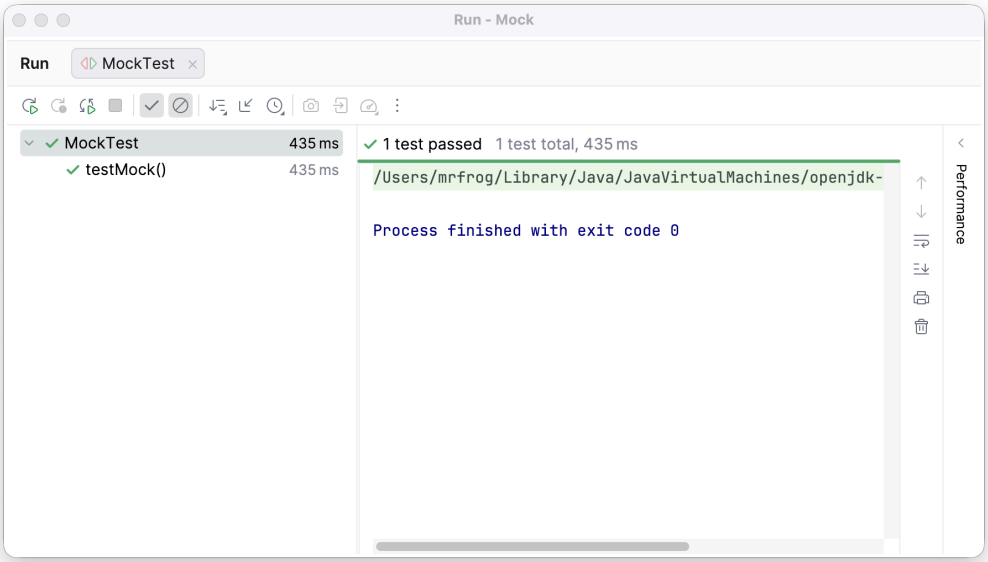
```java
import org.junit.jupiter.api.Test;
import static org.mockito.Mockito.*;

public class MockTest {
    @Test
    public void testMock(){
        UserRepository mockRepo = mock(UserRepository.class);
        UserService userService = new UserService(mockRepo);
        User user = new User("A01","Budiono");

        userService.register(user);

        verify(mockRepo, times(1)).save(user);

        verify(mockRepo, never()).findById(anyString());
    }
}
```

Listing .4: MockTest.java

Pada pengujian di atas, objek `mockRepo` dibuat sebagai mock dari `UserRepository`. Setelah metode `register(user)` dipanggil, dilakukan dua verifikasi interaksi:

1. `verify(mockRepo, times(1)).save(user)` — memastikan bahwa metode `save` dipanggil tepat satu kali dengan argumen `user`.

2. `verify(mockRepo, never()).findById(anyString())` — memastikan bahwa metode `findById` *tidak pernah* dipanggil selama proses registrasi.



Gambar 1: Hasil eksekusi MockTest

2.1.3 Spy Test

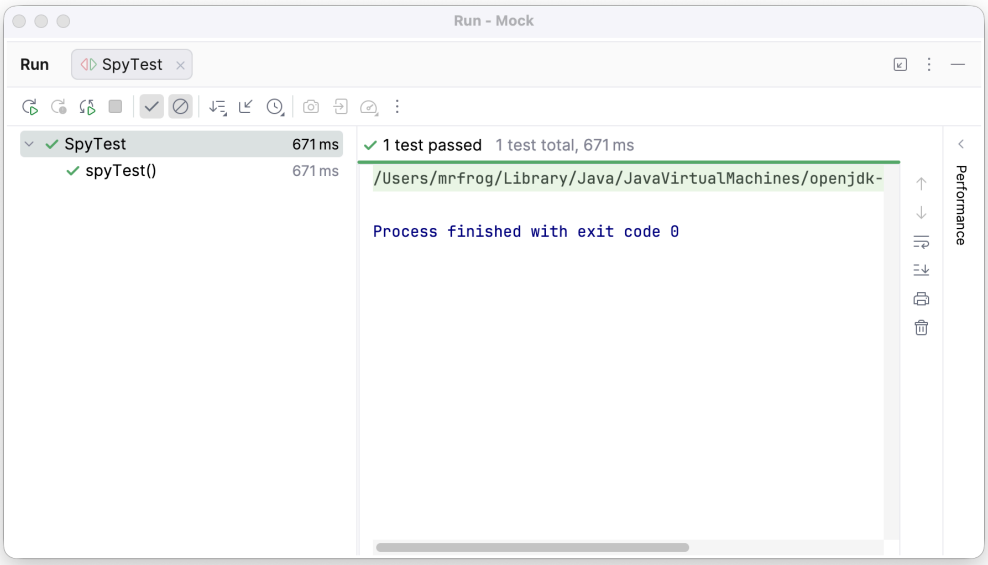
Spy adalah pembungkus (*wrapper*) dari objek nyata. Berbeda dengan Mock, Spy menjalankan implementasi asli dari objek yang dibungkusnya, kecuali pada metode yang secara eksplisit di-*stub*. Spy cocok digunakan ketika ingin melakukan verifikasi interaksi sekaligus mempertahankan perilaku nyata dari objek.

```java
import org.junit.jupiter.api.Test;
import java.util.ArrayList;
import java.util.List;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.spy;
import static org.mockito.Mockito.verify;

public class SpyTest {
    @Test
    public void spyTest(){
        // 1. setup spy
        List<String> realList = new ArrayList<>();
        List<String> spyList  = spy(realList);

        // 2. execution
        spyList.add("A01");
        spyList.add("A02");

        // 3. state verification
        assertEquals(2, spyList.size());
        assertEquals("A01", spyList.get(0));

        // 4. interaction verification
        verify(spyList).add("A02");
        verify(spyList).add("A01");
    }
}
```

Listing .5: SpyTest.java

Langkah-langkah pengujian Spy:

1. **Setup Spy** — dibuat objek `ArrayList` nyata, kemudian dibungkus dengan `spy()`.
2. **Eksekusi** — dua elemen ditambahkan ke `spyList`. Karena ini Spy, metode `add` yang asli benar-benar dieksekusi.
3. **Verifikasi State** — memastikan ukuran list adalah 2 dan elemen pertama adalah `"A01"`.
4. **Verifikasi Interaksi** — memastikan `add("A01")` dan `add("A02")` memang pernah dipanggil.



Gambar 2: Hasil eksekusi SpyTest

2.1.4 Stub Test

Stub adalah objek tiruan yang dikonfigurasi untuk mengembalikan nilai tertentu saat metode tertentu dipanggil. Stub digunakan untuk *verifikasi state*, yaitu memeriksa kebenaran nilai yang dihasilkan oleh unit yang sedang diuji, tanpa memperhatikan bagaimana objek dependensinya dipanggil.

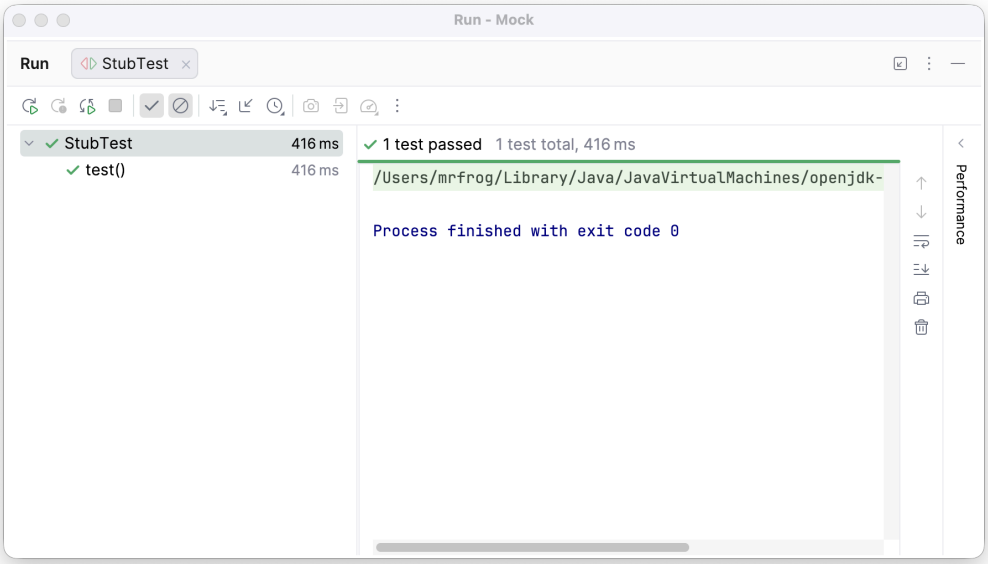
```java
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.mock;
import static org.mockito.Mockito.when;

public class StubTest {
    @Test
    public void test() {
        // 1. setup stub (tiruan)
        UserRepository stubRepo = mock(UserRepository.class);

        // 2. setup stubing
        User fakeUser = new User("A01", "Budiono");
```

```java
        when(stubRepo.findById("A01")).thenReturn(fakeUser);

        // 3. service test
        UserService service = new UserService(stubRepo);
        String name = service.getUserName("A01");

        assertEquals(name, fakeUser.name);
    }
}
```

Listing .6: StubTest.java

Pada pengujian Stub:

1. `stubRepo` dibuat sebagai mock dari `UserRepository`.
2. Dikonfigurasi agar `findById("A01")` mengembalikan objek `fakeUser`.
3. `UserService.getUserName("A01")` dipanggil, dan hasilnya diverifikasi sama dengan `fakeUser.name`, yaitu `"Budiono"`.



Gambar 3: Hasil eksekusi StubTest

2.2 Tugas

Pada bagian tugas, dibuat skenario pengujian untuk sistem perpustakaan sederhana yang terdiri dari kelas `Book`, antarmuka `BookRepository` dan `NotificationService`, serta kelas `BookController`. Pengujian dilakukan menggunakan anotasi Mockito (`@Mock`, `@InjectMocks`, `@ExtendWith`).

2.2.1 Kelas-Kelas Book

Kelas `Book` merepresentasikan buku dengan atribut `id`, `title`, dan `available`.

```java
package org.example;

public class Book {
    private String id;
    private String title;
    private boolean available;

    public Book(String id, String title, boolean available) {
        this.id = id;
        this.title = title;
        this.available = available;
    }

    public String getId()       { return id; }
    public String getTitle()    { return title; }
    public boolean isAvailable(){ return available; }
}
```

Listing .7: Book.java

`BookRepository` adalah antarmuka yang mendefinisikan operasi `findById`, `findAll`, dan `peminjaman`.

```java
package org.example;

import java.util.List;
import java.util.Optional;

public interface BookRepository {
    Optional<Book> findById(String id);
    List<Book> findAll();
    void peminjaman(String title, String username);
}
```

Listing .8: BookRepository.java

`NotificationService` adalah antarmuka untuk mengirim notifikasi kepada pengguna.

```java
package org.example;

public interface NotificationService {
    void sendNotification(String username, String title);
}
```

Listing .9: NotificationService.java

`BookController` adalah kelas utama yang mengandung logika bisnis: pengecekan ketersediaan buku dan alur peminjaman.

```java
package org.example;

import java.util.List;

public class BookController {
    private final BookRepository repository;
    private final NotificationService notificationService;

    public BookController(BookRepository repository,
                          NotificationService notificationService) {
        this.repository = repository;
        this.notificationService = notificationService;
    }

    public boolean checkAvailability(String id) {
        return repository.findById(id)
                .map(Book::isAvailable)
                .orElse(false);
    }

    public void borrowBook(String id, String username) {
        List<Book> books = repository.findAll(); // Urutan 1

        Book book = repository.findById(id)
                .orElseThrow(() -> new RuntimeException("Book not found"));

        if (book.isAvailable()) {
            repository.peminjaman(book.getTitle(), username); // Urutan 2
            notificationService.sendNotification(username,
                                                 book.getTitle()); // Urutan 3
        }
    }
}
```

Listing .10: BookController.java

2.2.2 BookTest

Kelas `BookTest` berisi tiga skenario pengujian yang menggabungkan teknik **Stub** (untuk men-*stub* nilai kembalian) dan **Mock** (untuk verifikasi interaksi dan urutan pemanggilan). Berikut adalah struktur awal kelas beserta deklarasi dependensi yang di-*inject* menggunakan anotasi Mockito.

```java
@ExtendWith(MockitoExtension.class)
public class BookTest {

    @Mock
    private BookRepository bookRepository;

    @Mock
    private NotificationService notificationService;

    @InjectMocks
    private BookController bookController;
}
```

Listing .11: BookTest.java – Deklarasi Kelas dan Anotasi

Penjelasan deklarasi:

- `@ExtendWith(MockitoExtension.class)` — mengaktifkan ekstensi Mockito pada JUnit 5 sehingga anotasi `@Mock` dan `@InjectMocks` diproses otomatis sebelum setiap test.
- `@Mock` — membuat objek mock untuk `BookRepository` dan `NotificationService` tanpa implementasi nyata.
- `@InjectMocks` — membuat instansi `BookController` dan menyuntikkan mock di atas ke dalamnya secara otomatis.

2.2.2.1 Test 1: testCheckAvailability_Success

Test ini memverifikasi bahwa `checkAvailability` mengembalikan `true` ketika buku dengan ID yang diminta tersedia.

```java
@Test
void testCheckAvailability_Success() {
    Book book = new Book("B01", "Clean Code", true);
    when(bookRepository.findById("B01")).thenReturn(Optional.of(book));

    boolean available = bookController.checkAvailability("B01");

    assertTrue(available);
    verify(bookRepository).findById("B01");
}
```

Listing .12: BookTest.java – testCheckAvailability_Success

Langkah-langkah pengujian:

1. **Stub** — `findById("B01")` dikonfigurasi agar mengembalikan `Optional` berisi buku dengan `available = true`.
2. **Eksekusi** — `checkAvailability("B01")` dipanggil pada `bookController`.
3. **Verifikasi State** — `assertTrue(available)` memastikan nilai kembalian adalah `true`.
4. **Verifikasi Interaksi** — `verify(bookRepository).findById("B01")` memastikan metode `findById` dipanggil tepat satu kali.

2.2.2.2 Test 2: testBorrowBook_WhenAvailable_ShouldCallPeminjamanAndNotify

Test ini memverifikasi bahwa ketika buku tersedia, alur peminjaman berjalan dengan urutan yang benar: `findAll` → `peminjaman` → `sendNotification`.

```java
@Test
void testBorrowBook_WhenAvailable_ShouldCallPeminjamanAndNotify() {
    String id = "B01";
    String username = "bangjan";
    Book book = new Book(id, "Clean Code", true);

    when(bookRepository.findAll())
            .thenReturn(Collections.singletonList(book));
    when(bookRepository.findById(id))
            .thenReturn(Optional.of(book));
```

```java

    bookController.borrowBook(id, username);

    InOrder inOrder = inOrder(bookRepository, notificationService);
    inOrder.verify(bookRepository).findAll();
    inOrder.verify(bookRepository)
            .peminjaman("Clean Code", username);
    inOrder.verify(notificationService)
            .sendNotification(username, "Clean Code");
}
```

Listing .13: BookTest.java – testBorrowBook_WhenAvailable

Langkah-langkah pengujian:

1. **Stub** — `findAll()` di-*stub* mengembalikan list berisi satu buku, dan `findById(id)` mengembalikan buku yang sama dengan `available = true`.

2. **Eksekusi** — `borrowBook(id, username)` dipanggil, memicu seluruh alur peminjaman di dalam `BookController`.

3. **Verifikasi Urutan (InOrder)** — menggunakan `InOrder` untuk memastikan urutan pemanggilan tepat: pertama `findAll()`, kemudian `peminjaman("Clean Code", username)`, terakhir `sendNotification(username, "Clean Code")`.

2.2.2.3 Test 3: testBorrowBook_WhenNotAvailable_ShouldNotCallPeminjaman

Test ini memverifikasi bahwa ketika buku tidak tersedia (`available = false`), metode `peminjaman` dan `sendNotification` tidak dipanggil sama sekali.

```java
@Test
void testBorrowBook_WhenNotAvailable_ShouldNotCallPeminjaman() {
    String id = "B02";
    String username = "bangjan";
    Book book = new Book(id, "Design Patterns", false);

    when(bookRepository.findAll())
            .thenReturn(Collections.singletonList(book));
    when(bookRepository.findById(id))
            .thenReturn(Optional.of(book));

    bookController.borrowBook(id, username);

    verify(bookRepository).findAll();
    verify(bookRepository, never())
            .peminjaman(anyString(), anyString());
    verify(notificationService, never())
            .sendNotification(anyString(), anyString());
}
```

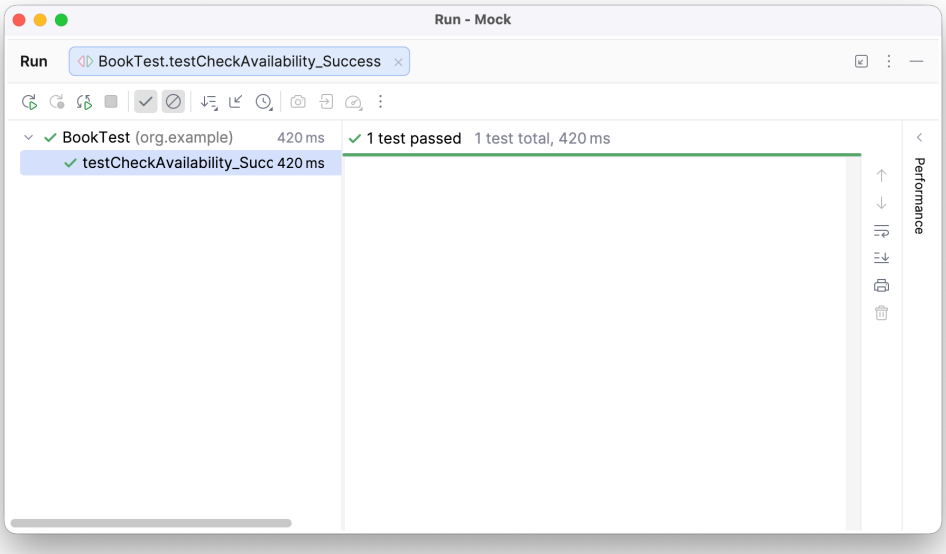
Listing .14: BookTest.java – testBorrowBook_WhenNotAvailable

Langkah-langkah pengujian:

1. **Stub** — `findAll()` dan `findById(id)` di-*stub* dengan buku yang memiliki `available = false`.

2. **Eksekusi** — `borrowBook(id, username)` dipanggil. Karena buku tidak tersedia, blok `if (book.isAvailable())` tidak dieksekusi.

3. **Verifikasi Interaksi Negatif** — `verify(..., never())` memastikan bahwa `peminjaman` dan `sendNotification` *tidak pernah* dipanggil, mengonfirmasi bahwa logika kondisional berfungsi dengan benar.



Gambar 4: Hasil eksekusi BookTest

Resources

https://github.com/januarsyah901/PPPL/tree/9d622d374dcf5c9e04c560607ba4e59049f3681a/Pertemuan%204

Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan beberapa hal berikut:

Berdasarkan praktikum yang telah dilakukan, pengujian dengan Mockito menunjukkan bahwa **Mock** efektif untuk memverifikasi *interaksi* antara unit yang diuji dan dependensinya, misalnya untuk memastikan suatu metode dipanggil atau tidak dipanggil dengan argumen tertentu. Sementara itu, **Spy** membungkus objek nyata sehingga implementasi asli tetap berjalan, sehingga pendekatan ini sesuai ketika pengujian membutuhkan verifikasi interaksi sekaligus perilaku asli objek. Di sisi lain, **Stub** berperan untuk mengatur nilai kembalian metode tertentu agar pengujian berfokus pada *verifikasi state* hasil eksekusi, bukan pada mekanisme pemanggilan dependensi. Secara keseluruhan, kombinasi Mock dan Stub seperti pada `BookTest` menghasilkan pengujian yang lebih komprehensif karena mampu mencakup verifikasi nilai, verifikasi interaksi, serta verifikasi urutan pemanggilan metode menggunakan `InOrder`.

Daftar Pustaka

[1] Mockito Project, "Mockito — tasty mocking framework for unit tests in Java". [Online]. Tersedia: https://site.mockito.org/.

[2] JUnit Team, "JUnit 5 User Guide". [Online]. Tersedia: https://junit.org/junit5/docs/current/user-guide/.

[3] Martin Fowler, "Mocks Aren't Stubs", 2007. [Online]. Tersedia: https://martinfowler.com/articles/mocksArentStubs.html.